

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



SOICT

Báo Cáo Nhập môn An toàn thông tin

Đề tài: Các cơ chế phát hiện xâm nhập mạng

Học phần: Nhập môn An toàn thông tin

Mã học phần: IT4015

Giảng viên hướng dẫn: PGS. TS. Nguyễn Linh Giang

Sinh viên thực hiện:	Nguyễn Đình Tuấn Minh	20230048
	Vũ Đức Tâm	20230064
	Nguyễn Gia Khánh	202416803
	Nguyễn Đăng Cao Tuấn	202400119

Hà Nội, Ngày 24 tháng 6 năm 2026

Mục lục

Lời nói đầu	3
1 Giới thiệu đề tài	4
1.1 Bối cảnh và lý do chọn đề tài	4
1.2 Mục tiêu đề tài	4
1.3 Đối tượng và phạm vi nghiên cứu	4
1.4 Phương pháp thực hiện	5
1.5 Bố cục báo cáo	5
2 Cơ sở lý thuyết	6
2.1 IDS, IPS và NIDS	6
2.2 Phát hiện theo chữ ký và IOC	7
2.3 Phát hiện theo hành vi	7
2.4 Các kiểu tấn công trong phạm vi bài tập	7
2.4.1 Quét cổng	7
2.4.2 ICMP Flood và SYN Flood	8
2.4.3 DNS Tunneling	8
2.5 Giới hạn lý thuyết liên quan đến triển khai	8
3 Kiến trúc và Triển khai Hệ thống NIDS	9
3.1 Tổng quan kiến trúc	9
3.2 Thu thập và chuẩn hóa gói tin	9
3.2.1 Capture	9
3.2.2 PacketEvent	10
3.3 Detection Engine	10
3.4 Luật phát hiện tích hợp	11
3.5 Luật chữ ký dạng Suricata-style subset	11
3.6 Lưu trữ và dashboard	12
4 Kịch bản Thử nghiệm và Đánh giá	13
4.1 Môi trường thử nghiệm	13
4.2 Khởi động hệ thống	13
4.3 Các kịch bản kiểm thử	14
4.3.1 Quét cổng bằng nmap	14
4.3.2 ICMP Ping Flood	14
4.3.3 TCP SYN Flood	14
4.3.4 DNS tunneling	14
4.3.5 DNS signature rule	14
4.4 Đánh giá kết quả	15

5	Kết luận và Hướng phát triển	16
5.1	Kết quả đạt được	16
5.2	Hạn chế	16
5.3	Hướng phát triển	17
5.4	Kết luận	17
	Tài liệu tham khảo	18

Lời nói đầu

Trong kỷ nguyên số hóa, hệ thống mạng là nền tảng cho hầu hết hoạt động của tổ chức và doanh nghiệp. Cùng với lợi ích đó, các rủi ro an ninh mạng cũng xuất hiện thường xuyên hơn, đặc biệt là các hành vi thăm dò, khai thác dịch vụ và truyền dữ liệu bất thường qua những giao thức hợp lệ.

Trong bối cảnh đó, các biện pháp phòng thủ tĩnh như tường lửa vẫn cần thiết nhưng chưa đủ để quan sát nội dung và hành vi của lưu lượng mạng. Hệ thống phát hiện xâm nhập mạng (NIDS) giúp bổ sung lớp giám sát, tạo cảnh báo khi lưu lượng có dấu hiệu bất thường hoặc khớp với mẫu tấn công đã biết.

Trong khuôn khổ học phần **IT4015 - Nhập môn An toàn thông tin**, nhóm em lựa chọn thực hiện đề tài: **"Các cơ chế phát hiện xâm nhập mạng"**. Trọng tâm của bài tập lớn là xây dựng một NIDS nhỏ gọn bằng Python để minh họa quy trình thu thập gói tin, chuẩn hóa dữ liệu, phát hiện theo hành vi, phát hiện theo chữ ký và hiển thị cảnh báo.

Mục tiêu của báo cáo là trình bày ngắn gọn cơ sở lý thuyết cần thiết, mô tả kiến trúc hệ thống đã triển khai, kiểm thử trong môi trường lab và chỉ ra các giới hạn còn tồn tại.

Mặc dù đã nỗ lực tìm hiểu và tổng hợp tài liệu, song do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, bản báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự nhận xét và góp ý quý báu từ thầy để kiến thức của mình được hoàn thiện hơn. Em xin chân thành cảm ơn!

Hà Nội, ngày 24 tháng 06 năm 2026

Nhóm sinh viên thực hiện

Chương 1

Giới thiệu đề tài

1.1 Bối cảnh và lý do chọn đề tài

Trong các hệ thống mạng hiện đại, tường lửa chỉ là một lớp bảo vệ ban đầu. Nhiều cuộc tấn công vẫn đi qua các cổng hợp lệ như HTTP, DNS hoặc HTTPS, sau đó khai thác điểm yếu ở tầng ứng dụng hoặc tạo ra các hành vi bất thường trong mạng nội bộ. Vì vậy, hệ thống phát hiện xâm nhập mạng (Network Intrusion Detection System – NIDS) là một thành phần quan trọng để giám sát lưu lượng, phát hiện dấu hiệu tấn công và hỗ trợ quản trị viên phản ứng kịp thời.

Đề tài này tập trung xây dựng một NIDS nhỏ gọn bằng Python nhằm minh họa các thành phần cốt lõi của một hệ thống phát hiện xâm nhập: thu thập gói tin, chuẩn hóa dữ liệu, phát hiện theo hành vi, phát hiện theo chữ ký, lưu trữ cảnh báo và hiển thị qua dashboard. Mục tiêu chính là phục vụ học tập và trình diễn kỹ thuật, không hướng tới thay thế các hệ thống sản xuất như Suricata hoặc Zeek.

1.2 Mục tiêu đề tài

Bài tập đặt ra các mục tiêu sau:

- **Thu thập và chuẩn hóa gói tin:** đọc lưu lượng từ tệp PCAP hoặc giao diện mạng trực tiếp bằng Scapy, sau đó chuyển đổi thành cấu trúc sự kiện thống nhất.
- **Phát hiện theo hành vi:** triển khai các luật dựa trên cửa sổ thời gian để nhận diện quét cổng, ICMP flood, TCP SYN flood và dấu hiệu DNS tunneling.
- **Phát hiện theo chữ ký/IOC:** xây dựng bộ phân tích cú pháp cho một tập con cú pháp kiểu Suricata và kiểm tra các mẫu IOC như domain blacklist hoặc truy vấn DNS đã biết.
- **Ghi nhận và trực quan hóa cảnh báo:** lưu cảnh báo dưới dạng JSON Lines và hiển thị thống kê qua giao diện Flask.
- **Kiểm thử trong môi trường lab:** mô phỏng các luồng tấn công phổ biến bằng Kali Linux để đánh giá khả năng kích hoạt cảnh báo của hệ thống.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là hệ thống phát hiện xâm nhập mạng dạng thụ động. Hệ thống quan sát lưu lượng, tạo cảnh báo và phục vụ phân tích, nhưng không trực tiếp chặn gói tin hoặc ngắt phiên kết nối như một hệ thống IPS.

Phạm vi triển khai của bài tập lớn được giới hạn như sau:

- Hệ thống xử lý các trường thông tin phổ biến của IP, TCP, UDP, ICMP và DNS.
- Phần phát hiện hành vi sử dụng ngưỡng tĩnh và cửa sổ trượt thời gian; các ngưỡng này là giá trị lab/demo, cần hiệu chỉnh theo baseline từng mạng thực tế.
- Phần chữ ký hỗ trợ một *Suricata-style subset*, đủ cho tình huống demo khớp domain IOC mẫu trong truy vấn DNS.
- Hệ thống chưa giải mã TLS/HTTPS, chưa tái tạo đầy đủ TCP stream và chưa được tối ưu cho lưu lượng mạng tốc độ cao.

1.4 Phương pháp thực hiện

Đề tài được triển khai theo hướng kết hợp lý thuyết và thực nghiệm:

- Nghiên cứu các cơ chế IDS cơ bản, đặc biệt là phát hiện theo chữ ký và phát hiện theo hành vi.
- Thiết kế kiến trúc pipeline gồm các khối capture, parser, detection engine, storage và dashboard.
- Cài đặt từng module bằng Python, sử dụng Scapy cho xử lý gói tin và Flask cho giao diện web.
- Viết luật kiểm thử và kịch bản tấn công mô phỏng để kiểm tra kết quả cảnh báo.
- Đối chiếu kết quả với phạm vi đã đặt ra, đồng thời chỉ ra hạn chế còn tồn tại.

1.5 Bố cục báo cáo

Báo cáo gồm 5 chương:

- **Chương 1: Giới thiệu đề tài** – trình bày bối cảnh, mục tiêu, phạm vi và phương pháp thực hiện.
- **Chương 2: Cơ sở lý thuyết** – tóm tắt các khái niệm IDS/IPS, NIDS, phát hiện theo chữ ký và phát hiện theo hành vi.
- **Chương 3: Kiến trúc và triển khai hệ thống** – mô tả các module chính và cách dữ liệu đi qua hệ thống.
- **Chương 4: Kịch bản thử nghiệm và đánh giá** – trình bày môi trường lab, các kịch bản tấn công và kết quả quan sát.
- **Chương 5: Kết luận và hướng phát triển** – tổng kết kết quả đạt được, hạn chế và hướng cải tiến.

Chương 2

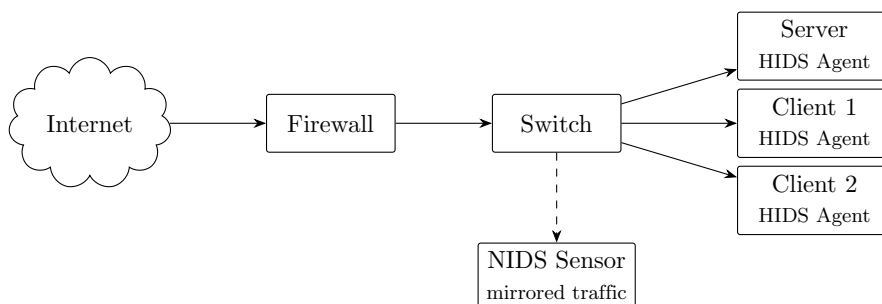
Cơ sở lý thuyết

2.1 IDS, IPS và NIDS

Hệ thống phát hiện xâm nhập (Intrusion Detection System – IDS) có nhiệm vụ thu thập và phân tích sự kiện bảo mật để phát hiện hành vi bất thường hoặc dấu hiệu tấn công. IDS thường hoạt động ở chế độ thụ động: hệ thống quan sát lưu lượng, ghi log và sinh cảnh báo, nhưng không trực tiếp can thiệp vào luồng dữ liệu.

Hệ thống ngăn chặn xâm nhập (Intrusion Prevention System – IPS) có phạm vi rộng hơn IDS vì được đặt inline trên đường truyền và có khả năng chặn gói tin, reset kết nối hoặc cập nhật luật tường lửa. Bài tập lớn này chỉ triển khai hướng IDS, tức là tập trung vào phát hiện và cảnh báo.

Theo nguồn dữ liệu giám sát, IDS có thể được chia thành hai nhóm chính:



Hình 2.1: Minh họa vị trí giám sát của NIDS và HIDS

Loại hệ thống	Nguồn dữ liệu	Đặc điểm
NIDS	Lưu lượng mạng tại switch, gateway hoặc interface giám sát.	Quan sát được nhiều máy trong cùng phân đoạn mạng, phù hợp phát hiện scan, flood và payload độc hại trong lưu lượng không mã hóa.
HIDS	Log, tiến trình, file và hành vi trên từng máy chủ hoặc endpoint.	Nhìn sâu vào trạng thái máy trạm nhưng cần cài agent trên từng host.

Bảng 2.1: Phân loại IDS theo nguồn dữ liệu giám sát

Hệ thống trong bài tập là một NIDS dạng thụ động: đọc PCAP hoặc sniff interface, phân tích gói tin và ghi cảnh báo ra tệp JSONL.

2.2 Phát hiện theo chữ ký và IOC

Phát hiện theo chữ ký (Signature-based Detection) so khớp lưu lượng với các mẫu đã biết. Mẫu có thể là chuỗi trong payload, biểu thức chính quy, truy vấn DNS, domain trong blacklist/IOC hoặc tổ hợp điều kiện trong header gói tin. Cách tiếp cận này phù hợp để nhận diện các dấu hiệu đã biết như tên miền độc hại, domain phục vụ kiểm thử, hoặc payload có cấu trúc đặc trưng.

Một luật Suricata/Snort-like thường gồm hai phần:

```
alert dns any any -> any 53 (msg:"Custom DNS Query: example.com";  
dns.query; content:"example.com"; nocase; sid:1000101; rev:1;)
```

- **Header luật:** xác định hành động, giao thức, địa chỉ IP, cổng và hướng lưu lượng.
- **Options:** mô tả điều kiện cần so khớp như `content`, `pcrc`, `dns.query`, `sid`, `priority`.

Ưu điểm của phát hiện theo chữ ký là cảnh báo rõ ràng và dễ kiểm chứng. Hạn chế là hệ thống chỉ phát hiện tốt các mẫu đã được mô tả trước; nếu payload bị mã hóa, bị làm rối hoặc chưa có rule phù hợp thì khả năng bỏ sót sẽ tăng.

2.3 Phát hiện theo hành vi

Phát hiện theo hành vi (Behavior-based Detection) quan sát chuỗi sự kiện theo thời gian thay vì chỉ xét một gói tin riêng lẻ. Trong bài tập này, cơ chế hành vi được triển khai bằng cửa sổ trượt thời gian. Hệ thống lưu các sự kiện gần nhất theo từng địa chỉ nguồn, loại bỏ các sự kiện đã quá hạn và kích hoạt cảnh báo khi số lượng hoặc đặc điểm sự kiện vượt ngưỡng.

Các ví dụ được triển khai:

- Một IP truy cập nhiều cổng TCP khác nhau trong thời gian ngắn có thể là quét cổng.
- Một IP gửi quá nhiều ICMP Echo Request có thể là ping flood.
- Một IP gửi quá nhiều gói TCP SYN có thể là SYN flood.
- Các truy vấn DNS có nhãn quá dài hoặc entropy cao có thể là dấu hiệu DNS tunneling.

Cách tiếp cận này không cần biết trước chính xác payload tấn công, nhưng phụ thuộc nhiều vào ngưỡng cấu hình. Trong prototype, các ngưỡng như 20 cổng/10 giây hoặc 100 gói/10 giây được chọn để phục vụ demo trong lab; khi áp dụng vào mạng thực tế cần đo baseline lưu lượng bình thường để giảm false positive. Ngưỡng quá thấp dễ tạo cảnh báo sai; ngưỡng quá cao có thể bỏ sót các cuộc tấn công chậm.

2.4 Các kiểu tấn công trong phạm vi bài tập

Báo cáo tập trung vào các tình huống có thể kiểm thử trực tiếp bằng hệ thống đã xây dựng.

2.4.1 Quét cổng

Quét cổng là bước thăm dò nhằm xác định dịch vụ đang mở trên máy mục tiêu. Dấu hiệu chính trong hệ thống là một địa chỉ nguồn gửi gói TCP tới nhiều cổng đích khác nhau trong cùng một cửa sổ thời gian.

2.4.2 ICMP Flood và SYN Flood

ICMP Flood tạo lượng lớn gói Echo Request để tiêu tốn băng thông hoặc tài nguyên xử lý. SYN Flood tạo nhiều yêu cầu TCP SYN chưa hoàn tất kết nối. Cả hai đều được phát hiện bằng bộ đếm theo nguồn trong cửa sổ thời gian.

2.4.3 DNS Tunneling

DNS có thể bị lợi dụng để truyền dữ liệu ra ngoài thông qua các nhãn dài, khó đọc, entropy cao. Trong bài tập lớn, DNS tunneling được nhận diện bằng dấu hiệu bất thường của chuỗi truy vấn trong cửa sổ thời gian. Các truy vấn khớp domain blacklist hoặc domain kiểm thử có thể được bổ sung bằng luật chữ ký tùy chỉnh, nhưng không nằm trong nhóm rule mặc định.

2.5 Giới hạn lý thuyết liên quan đến triển khai

Các cơ chế trên có một số giới hạn cần được hiểu đúng khi đánh giá kết quả:

- NIDS không nhìn thấy nội dung ứng dụng nếu lưu lượng được mã hóa bằng TLS mà không có cơ chế giải mã hợp lệ.
- Nếu không tái tạo TCP stream, chữ ký có thể bỏ sót payload bị chia nhỏ qua nhiều gói tin.
- Nếu không xử lý IP fragmentation, TCP segmentation và out-of-order packets, hệ thống có thể bỏ sót hoặc phân tích sai một số luồng dữ liệu.
- Cảnh báo hành vi phụ thuộc vào ngưỡng và môi trường mạng cụ thể.
- Các kỹ thuật encode, obfuscate hoặc chia nhỏ payload có thể né tránh signature matching đơn giản.
- Hệ thống demo dùng Python và Scapy phù hợp cho học tập, nhưng không đại diện cho hiệu năng của IDS triển khai trong mạng doanh nghiệp.

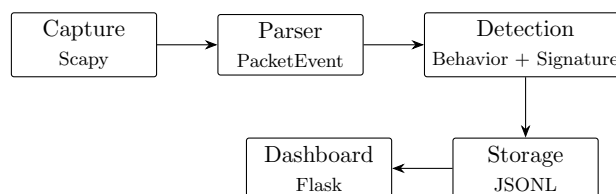
Chương 3

Kiến trúc và Triển khai Hệ thống NIDS

3.1 Tổng quan kiến trúc

Hệ thống NIDS được xây dựng bằng Python 3 và tổ chức theo mô hình pipeline. Mỗi gói tin sau khi được đọc từ PCAP hoặc interface mạng sẽ đi qua các bước: thu thập, chuẩn hóa, phát hiện, lưu trữ và hiển thị.

1. **Capture:** đọc gói tin bằng Scapy từ tệp PCAP hoặc giao diện mạng.
2. **Parser:** chuyển gói tin Scapy thành đối tượng `PacketEvent`.
3. **Detection Engine:** chạy luật hành vi/anomaly và luật chữ ký.
4. **Storage:** ghi cảnh báo ra tệp JSONL.
5. **Dashboard:** đọc dữ liệu cảnh báo và hiển thị thống kê bằng Flask.



Hình 3.1: Luồng dữ liệu của hệ thống NIDS

3.2 Thu thập và chuẩn hóa gói tin

3.2.1 Capture

Module `nids/capture.py` cung cấp hai cách lấy dữ liệu: đọc từ tệp PCAP và sniff trực tiếp trên interface. Với live capture, Scapy gọi callback cho từng gói tin; nếu parser chuyển đổi thành công thì sự kiện sẽ được đưa vào detection engine.

Đoạn mã 3.1: Live capture bằng Scapy

```
1 def sniff_live(interface: str, callback: PacketCallback, limit: int = 0) -> None
  :
2   from scapy.all import sniff
```

```
3
4 def handle_packet(packet: object) -> None:
5     event = packet_to_event(packet)
6     if event is not None:
7         callback(event)
8
9 sniff(iface=interface, prn=handle_packet, store=False, count=limit)
```

3.2.2 PacketEvent

Raw packet chứa nhiều trường phụ thuộc giao thức nên khó xử lý trực tiếp trong rule engine. Vì vậy, module `nids/parser.py` chuẩn hóa các trường cần thiết thành `PacketEvent`: thời gian, IP nguồn/đích, cổng nguồn/đích, giao thức, TCP flags, ICMP type, DNS query, payload text và các trường HTTP phổ biến.

Đoạn mã 3.2: Cấu trúc PacketEvent

```
1 @dataclass(frozen=True)
2 class PacketEvent:
3     timestamp: float
4     src_ip: str | None
5     dst_ip: str | None
6     src_port: int | None
7     dst_port: int | None
8     protocol: str
9     length: int
10    tcp_flags: str | None = None
11    icmp_type: int | None = None
12    dns_query: str | None = None
13    payload_text: str | None = None
14    http_method: str | None = None
15    http_uri: str | None = None
16    http_host: str | None = None
17    http_user_agent: str | None = None
```

Payload được giới hạn kích thước khi chuyển thành chuỗi để tránh tốn bộ nhớ và giảm chi phí so khớp. Đây là lựa chọn phù hợp cho demo, nhưng cũng là một giới hạn khi phân tích payload lớn hoặc payload bị chia nhỏ.

3.3 Detection Engine

Module `nids/engine.py` đóng vai trò điều phối. Mỗi `PacketEvent` được đếm vào thống kê giao thức, sau đó gửi đến `SlidingWindowRules`. Các cảnh báo sinh ra được ghi xuống `AlertStore`.

Đoạn mã 3.3: Luồng xử lý một sự kiện

```
1 def process_event(self, event: PacketEvent) -> list[Alert]:
2     self.packet_count += 1
3     self.protocol_counts[event.protocol] += 1
4
5     alerts = self.rules.evaluate(event)
6     self.store.append_many(alerts)
7     self.alert_count += len(alerts)
8     return alerts
```

3.4 Luật phát hiện tích hợp

Các luật hành vi và anomaly được triển khai bằng cửa sổ trượt theo thời gian. Hệ thống lưu các sự kiện gần nhất theo IP nguồn và loại bỏ dữ liệu đã nằm ngoài khung thời gian cấu hình. Các ngưỡng trong bảng là giá trị lab/demo, chưa phải khuyến nghị production.

Đoạn mã 3.4: Loại bỏ sự kiện cũ trong cửa sổ thời gian

```
1 @staticmethod
2 def _drop_old_times(window: deque[float], now: float, seconds: int) -> None:
3     while window and now - window[0] > seconds:
4         window.popleft()
```

Các luật tích hợp gồm:

Rule ID	Attack	Nhóm phát hiện	Điều kiện mặc định	Severity
RULE-001	Port Scan	Behavior-based	≥ 20 TCP destination ports từ một source trong 10 giây.	Medium
RULE-002	ICMP Ping Flood	Behavior-based	≥ 100 ICMP request packets trong 10 giây.	High
RULE-003	TCP SYN Flood	Behavior-based	≥ 100 TCP SYN packets without ACK trong 10 giây.	High
RULE-004	DNS Tunneling	Behavior/ anomaly-based	≥ 5 DNS query đáng ngờ từ cùng source trong 30 giây.	Medium

Bảng 3.1: Các detection rules tích hợp trong prototype

Với port scan, prototype chỉ đếm các probe flags như SYN, FIN, NULL và Xmas. Xmas scan tương ứng FIN + PSH + URG. ACK scan có thể tạo false positive nếu không có state tracking đầy đủ, vì vậy phần phát hiện ACK scan chỉ nên xem là minh họa trong prototype.

3.5 Luật chữ ký dạng Suricata-style subset

Ngoài luật hành vi, hệ thống hỗ trợ một tập con cú pháp kiểu Suricata/Snort. Người dùng có thể truyền tệp luật qua tham số `-rules`. Trong lần thử nghiệm được lưu ở `data/alerts.jsonl`, luật chữ ký được kích hoạt là luật DNS kiểm tra truy vấn chứa `example.com`:

Đoạn mã 3.5: Luật chữ ký dùng trong demo

```
1 alert dns any any -> any 53 (msg:"Custom DNS Query: example.com"; dns.query;
   content:"example.com"; nocase; classtype:policy-violation; priority:3; sid
   :1000101; rev:1;)
```

Mục tiêu của phần này là minh họa cách một signature engine hoạt động: phân tích header luật, chọn buffer cần kiểm tra, sau đó so khớp nội dung hoặc biểu thức chính quy. Đây là *Suricata-style subset*, không phải engine tương thích đầy đủ với Suricata/Snort.

Subset prototype hỗ trợ các thành phần cơ bản như header luật, `content`, `pcre`, `nocase`, một số sticky buffer DNS/payload và threshold đơn giản. Các tính năng chưa hỗ trợ đầy đủ gồm `depth/offset`, `distance/within`, `byte_test`, `flowbits`, TCP stream inspection, `file_data` và tối ưu `fast_pattern`. Tùy chọn `flow` hiện chỉ được kiểm tra ở mức hướng cơ bản, chưa phải stateful TCP flow tracking như IDS production.

3.6 Lưu trữ và dashboard

Cảnh báo được biểu diễn bằng **Alert** và ghi dưới dạng JSON Lines. Mỗi dòng là một cảnh báo độc lập nên hệ thống có thể append liên tục mà không cần đọc lại toàn bộ tệp.

Dashboard trong `dashboard/app.py` đọc tệp cảnh báo, cache theo thời gian sửa đổi của file và hiển thị:

- tổng số cảnh báo,
- số cảnh báo theo mức độ nghiêm trọng,
- các IP nguồn xuất hiện nhiều nhất,
- danh sách cảnh báo gần nhất.

Dashboard phục vụ quan sát và trình diễn kết quả; phần phản ứng tự động như chặn IP hoặc cập nhật tường lửa chưa nằm trong phạm vi bài tập.

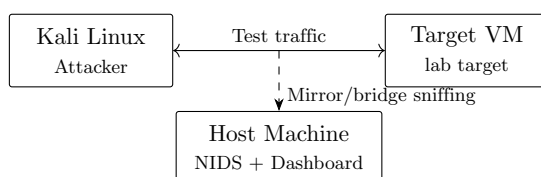
Chương 4

Kịch bản Thử nghiệm và Đánh giá

4.1 Môi trường thử nghiệm

Việc kiểm thử được thực hiện trong môi trường lab ảo hóa cô lập để tránh ảnh hưởng đến mạng thật. Các lệnh tạo traffic chỉ dùng trong lab/VM có kiểm soát và không chạy trên mạng thật nếu không có quyền. Mô hình gồm ba thành phần:

- **Host Machine:** chạy NIDS engine và Flask dashboard.
- **Kali Linux VM:** đóng vai trò máy tạo lưu lượng kiểm thử bằng `nmap`, `hping3` và `dig`.
- **Target VM:** nhận lưu lượng scan, flood và DNS trong môi trường lab.



Hình 4.1: Mô hình lab kiểm thử

Trong mô hình này, NIDS sniff traffic trên interface mirror/bridge của lab. Nếu dùng SPAN/TAP hoặc bridge trong môi trường thật, cần lưu ý khả năng mất packet hoặc asymmetric routing làm giảm độ đầy đủ của dữ liệu quan sát.

4.2 Khởi động hệ thống

Trên host, dashboard được chạy ở một terminal:

Đoạn mã 4.1: Khởi động dashboard

```
1 python3 dashboard/app.py
```

NIDS engine được chạy ở terminal khác. Interface cần thay bằng interface thực tế kết nối các máy ảo:

Đoạn mã 4.2: Khởi động NIDS engine

```
1 sudo .venv/bin/python main.py --iface <YOUR_INTERFACE> --rules custom_test.rules --limit 0
```

Khi dùng PCAP thay cho live capture, có thể chạy:

Đoạn mã 4.3: Phân tích PCAP

```
1 python3 main.py --pcap sample.pcap --rules custom_test.rules
```

4.3 Các kịch bản kiểm thử

4.3.1 Quét cổng bằng nmap

Kịch bản sử dụng **nmap** để gửi SYN tới nhiều cổng đích trong thời gian ngắn:

```
1 nmap -sS -p 21,22,23,25,53,80,110,135,139,143,443,445,\
2 993,995,1433,1723,3306,3389,5900,8080,8443,8888,9090,\
3 9200,9300 --scan-delay 30ms --max-retries 1 -T3 -n --open <TARGET_IP>
```

Kết quả mong đợi là cảnh báo **RULE-001 Port Scan**, vì cùng một IP nguồn truy cập nhiều cổng TCP đích trong cửa sổ 10 giây.

4.3.2 ICMP Ping Flood

Kịch bản gửi nhiều gói ICMP Echo Request:

```
1 hping3 --icmp -c 120 -i u80000 <TARGET_IP>
```

Tùy chọn **-i u80000** tạo khoảng cách 80000 micro giây giữa các gói, giúp lưu lượng đỡ đột ngột nhưng vẫn đủ vượt ngưỡng demo ≥ 100 packets/10 giây trong lab. Kết quả mong đợi là cảnh báo **RULE-002 ICMP Ping Flood** khi bộ đếm ICMP request từ cùng nguồn vượt ngưỡng.

4.3.3 TCP SYN Flood

Kịch bản gửi nhiều gói SYN tới cổng 80:

```
1 hping3 -S -p 80 -c 120 -i u80000 <TARGET_IP>
```

Tùy chọn **-S** tạo TCP SYN without ACK, còn **-i u80000** giữ tốc độ ổn định hơn nhưng vẫn đủ cao cho threshold demo. Kết quả mong đợi là cảnh báo **RULE-003 TCP SYN Flood**. Trong phạm vi bài tập, hệ thống chỉ cảnh báo dựa trên số lượng gói SYN without ACK từ cùng nguồn, chưa kiểm tra trạng thái hoàn tất kết nối ở mức TCP stream đầy đủ.

4.3.4 DNS tunneling

Kịch bản DNS tạo các truy vấn có nhãn dài/entropy cao.

```
1 dig aGVsbG93b3JsZGh1bGxvd2...tunnel.example.com
```

Kết quả mong đợi là **RULE-004 DNS Tunneling Suspicion** khi có đủ số truy vấn bất thường trong cửa sổ thời gian.

Rule DNS tunneling có thể false positive với một số domain hợp lệ như CDN, tracking, DKIM, ACME challenge hoặc service discovery. Trong thực tế nên kết hợp thêm whitelist, NXDOMAIN rate, số lượng unique subdomain, qtype và reputation của base domain.

4.3.5 DNS signature rule

Kịch bản DNS bình thường và DNS tunneling cũng được dùng để kiểm tra luật chữ ký tùy chỉnh. Luật trong file **custom_test.rules** so khớp các truy vấn DNS chứa chuỗi **example.com**:

```
1 dig +short example.com @8.8.8.8
2 dig +short <LONG_LABEL>.tunnel.example.com @8.8.8.8
```

Kết quả mong đợi là cảnh báo signature-based **1000101 Custom DNS Query: example.com**. Luật này dùng `dns.query` và `content` để minh họa cơ chế nạp rule tùy chỉnh dạng *Suricata-style subset*.

4.4 Đánh giá kết quả

Trong môi trường lab, các kịch bản trên kiểm tra được các nhóm phát hiện chính:

- Luật hành vi phát hiện các mẫu dựa trên tần suất và số lượng sự kiện trong thời gian ngắn.
- Luật signature phát hiện chuỗi đặc trưng trong truy vấn DNS.
- Dashboard hiển thị được số lượng cảnh báo, mức độ nghiêm trọng, IP nguồn và danh sách cảnh báo gần nhất.

Số liệu dưới đây được lấy trực tiếp từ file `data/alerts.jsonl`. File này lưu 14 cảnh báo thực nghiệm trong lần chạy lab: 3 cảnh báo behavior, 1 cảnh báo anomaly và 10 cảnh báo signature DNS.

Scenario	Expected alert	Observed alert	Evidence chính	Ghi chú
Port scan	RULE-001	1 alert	unique ports = 20, count = 20	Vượt ngưỡng scan trong cửa sổ 10 giây.
ICMP flood	RULE-002	1 alert	icmp packets = 100	Trigger đúng ngưỡng demo.
SYN flood	RULE-003	1 alert	syn packets = 100, dst_port = 80	SYN without ACK.
DNS tunneling	RULE-004	1 alert	suspicious queries = 5, max label entropy = 4.276, max label length = 54	≥ 5 suspicious queries/30 giây.
DNS signature	1000101	10 alerts	dns.query chứa example.com	Gồm truy vấn nền và các truy vấn tunnel.example.com.

Bảng 4.1: Bảng kết quả thực nghiệm từ file alert đã lưu

Kết quả này xác nhận hệ thống đáp ứng mục tiêu học tập và trình diễn. Tuy nhiên, đánh giá này chỉ có giá trị trong phạm vi lab có kiểm soát. Để kết luận về độ chính xác trong mạng thực tế cần có bộ dữ liệu lớn hơn, lưu lượng nền đa dạng hơn, kiểm thử false positive/false negative và đo hiệu năng xử lý gói tin.

Chương 5

Kết luận và Hướng phát triển

5.1 Kết quả đạt được

Bài tập đã xây dựng được một hệ thống NIDS nhỏ gọn phục vụ học tập và trình diễn. Hệ thống hoàn chỉnh các thành phần chính của một pipeline phát hiện xâm nhập mạng:

- Đọc lưu lượng từ PCAP hoặc live interface bằng Scapy.
- Chuẩn hóa packet thành cấu trúc `PacketEvent`.
- Phát hiện hành vi/anomaly bằng cửa sổ trượt thời gian với các ngưỡng lab/demo.
- Phân tích một *Suricata-style subset* để phát hiện chữ ký trong HTTP, DNS và payload.
- Tách domain blacklist khỏi nhóm rule mặc định; nếu cần phát hiện domain cụ thể thì nạp bằng luật chữ ký tùy chỉnh.
- Ghi cảnh báo ra JSON Lines và hiển thị qua dashboard Flask.
- Có kịch bản demo để kiểm tra các rule chính trong môi trường lab.

Kết quả quan trọng nhất của bài tập lớn là làm rõ cách các thành phần IDS phối hợp với nhau: capture cung cấp dữ liệu, parser chuẩn hóa dữ liệu, rule engine tạo cảnh báo, storage lưu lại kết quả và dashboard hỗ trợ quan sát.

5.2 Hạn chế

Hệ thống hiện tại vẫn có một số hạn chế kỹ thuật:

- **Chỉ phát hiện, chưa ngăn chặn:** hệ thống không chặn gói tin, không reset kết nối và không tự động cập nhật tường lửa.
- **Chưa xử lý lưu lượng mã hóa:** nội dung HTTPS/TLS không thể được kiểm tra bằng chữ ký nếu không có cơ chế giải mã hợp lệ.
- **Chưa tái tạo TCP stream đầy đủ:** payload bị chia nhỏ qua nhiều gói tin có thể làm giảm hiệu quả của signature matching.
- **Chưa xử lý đầy đủ fragmentation/segmentation:** IP fragmentation, TCP segmentation và out-of-order packets có thể làm sai lệch dữ liệu đầu vào của rule engine.
- **Dễ bị evasion với payload biến đổi:** payload bị encode, obfuscate hoặc chia nhỏ có thể vượt qua rule content đơn giản.

- **Phụ thuộc visibility của điểm giám sát:** SPAN/TAP có thể mất packet hoặc không nhìn thấy đủ hai chiều lưu lượng khi có asymmetric routing.
- **Hiệu năng còn giới hạn:** Python và Scapy phù hợp cho demo, nhưng chưa phù hợp với môi trường có lưu lượng lớn nếu không tối ưu thêm.
- **Phụ thuộc vào ngưỡng và luật:** chất lượng cảnh báo phụ thuộc vào cấu hình threshold, baseline từng mạng và độ đầy đủ của tập luật chữ ký.

5.3 Hướng phát triển

Các hướng cải tiến phù hợp với kiến trúc hiện tại gồm:

- Bổ sung IP defragmentation và TCP stream reassembly để kiểm tra payload ở mức phiên thay vì từng gói riêng lẻ.
- Bổ sung stateful protocol analysis cho TCP/HTTP/DNS và mở rộng parser rule theo hướng Snort/Suricata-like subset rõ ràng hơn.
- Thêm whitelist/baseline học từ traffic bình thường để giảm false positive.
- Thêm correlation engine để gom nhiều alert như port scan, flood hoặc HTTP SQLi thành incident có timeline.
- Thêm cơ chế cấu hình rule và threshold qua tệp YAML hoặc giao diện dashboard.
- Đánh giá trên PCAP lớn hoặc dataset công khai và báo cáo precision, recall, false positive rate.
- Đo hiệu năng trên PCAP lớn và tối ưu luồng xử lý để giảm packet drop.
- Tích hợp phản ứng bán tự động như xuất IOC, webhook hoặc cập nhật danh sách chặn cho firewall trong môi trường lab.
- Xây dựng bộ kiểm thử với cả traffic bình thường và traffic tấn công để đánh giá false positive/false negative rõ ràng hơn.

5.4 Kết luận

Hệ thống đã đáp ứng mục tiêu chính của bài tập lớn: minh họa được một NIDS hoạt động từ khâu thu thập gói tin đến sinh cảnh báo và trực quan hóa kết quả. Dù chưa phải hệ thống production, bài tập cung cấp nền tảng thực hành tốt để hiểu các kỹ thuật phát hiện xâm nhập mạng và các giới hạn khi triển khai IDS trong thực tế.

Tài liệu tham khảo

Tài liệu tham khảo

- [1] Bộ môn Truyền thông và Mạng máy tính. *Bài giảng Nhập môn An toàn thông tin (IT4015)*.
- [2] William Stallings (2016). *Cryptography and Network Security: Principles and Practice* (7th Edition). Pearson Education.
- [3] James P. Anderson (1980). *Computer Security Threat Monitoring and Surveillance*. Technical report, James P. Anderson Co., Fort Washington, Pennsylvania.
- [4] Dorothy E. Denning (1987). *An Intrusion-Detection Model*. IEEE Transactions on Software Engineering, Vol. SE-13, No. 2, pp. 222-232.
- [5] Karen Scarfone, Peter Mell (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS)*. NIST Special Publication 800-94.
- [6] OISF. *Suricata User Guide*. Truy cập trực tuyến tại: <https://suricata.readthedocs.io/>.
- [7] Snort Project. *Snort 3 Rule Writing Guide*. Truy cập trực tuyến tại: <https://docs.snort.org/>.
- [8] Zeek Project. *Zeek Documentation*. Truy cập trực tuyến tại: <https://docs.zeek.org/>.
- [9] Scapy Project. *Scapy Documentation*. Truy cập trực tuyến tại: <https://scapy.readthedocs.io/>.
- [10] Gordon Lyon. *Nmap Network Scanning*. Truy cập trực tuyến tại: <https://nmap.org/book/>.
- [11] The MITRE Corporation. *MITRE ATT&CK Enterprise Matrix*. Truy cập trực tuyến tại: <https://attack.mitre.org/>.